

Universidad de Guayaquil

Facultad de Ciencias Matemáticas y Físicas

Carrera de Ingeniería de Software

Proyecto 1 parcial

Asignatura

Construcción del software

Autores

Sánchez Mosquera Stin Josue
Castro Rodríguez Marvin Alexander

Docente

PhD. Gilberto Fernando Castro Aguilar, MSc.

Periodo

CICLO I 2025-2026

1. Introducción

En la actualidad, la gestión del tiempo y la productividad personal representan un desafío cotidiano para estudiantes y profesionales. Si bien existen diversas aplicaciones de gestión de tareas en el mercado, la mayoría limita sus funcionalidades más útiles a modelos de suscripción de pago, lo que genera una barrera de acceso para una gran parte de los usuarios.

TaskHero / Tasks Quest surge como una propuesta alternativa: una aplicación web de gestión de tareas que incorpora un sistema de gamificación para motivar el uso constante y la formación de hábitos productivos. A través de experiencia, niveles y recompensas, los usuarios pueden desbloquear progresivamente funcionalidades avanzadas sin necesidad de pago, democratizando el acceso a herramientas de productividad de alto valor.

El sistema será desarrollado bajo principios modernos de ingeniería de software, aplicando Arquitectura Hexagonal, principios SOLID, comunicación en tiempo real mediante WebSockets, y buenas prácticas de seguridad. El backend será implementado en Java con Spring Boot, la base de datos en MySQL, y el frontend con JavaScript vanilla consumiendo la API REST mediante AJAX y jQuery.

2. Justificación

El desarrollo de TaskHero / Tasks Quest se justifica por las siguientes razones:

2.1 Problema identificado

Las aplicaciones de productividad más populares (Todoist, TickTick, Notion) colocan sus funciones más útiles detrás de muros de pago. Esto, combinado con interfaces que no generan motivación intrínseca, lleva al abandono temprano de la herramienta por parte del usuario.

2.2 Propuesta de valor

- Accesibilidad gratuita con funciones desbloqueables por mérito, no por pago.
- Sistema de gamificación que genera hábito mediante recompensas y rachas diarias.
- Comunicación en tiempo real que habilita la colaboración entre usuarios.
- Arquitectura escalable y mantenible aplicando principios profesionales de software.

2.3 Relevancia académica

El proyecto permite aplicar de manera integral los conocimientos de la materia Construcción de Software: diseño de arquitecturas limpias, patrones de diseño, pruebas, manejo de concurrencia con WebSockets, y gestión de un ciclo de desarrollo completo en un tiempo acotado.

3. Objetivos

3.1 Objetivo General

Desarrollar una aplicación web de gestión de tareas con sistema de gamificación, notificaciones y comunicación en tiempo real, aplicando principios modernos de arquitectura de software (Hexagonal Architecture, SOLID) y buenas prácticas de desarrollo seguro, en un plazo de un mes.

3.2 Objetivos Específicos

- Implementar un sistema de autenticación seguro con registro, inicio de sesión, edición de perfil y recuperación de contraseña.
- Desarrollar el módulo de gestión de tareas con creación, edición, eliminación, prioridades, fechas límite y categorías.
- Construir un sistema de notificaciones en tiempo real (WebSocket) y por correo electrónico para recordatorios y eventos del sistema.
- Diseñar e implementar el motor de gamificación: puntos de experiencia (XP), niveles, rachas diarias y desbloqueo progresivo de funcionalidades.
- Desarrollar el módulo de chat en tiempo real entre usuarios para colaboración y grupos de estudio.
- Exponer una API REST documentada desde Spring Boot que sea consumida por el frontend mediante AJAX/jQuery.
- Aplicar Arquitectura Hexagonal y principios SOLID en la estructura del backend.
- Garantizar un sistema seguro, con manejo de errores y tolerancia básica a fallos.

4. Alcance del Proyecto

4.1 Módulos incluidos

Módulo	Funcionalidades	Prioridad
Usuarios	Registro, login, edición de perfil, recuperación de contraseña, JWT	Alta
Gestión de tareas	Crear, editar, eliminar tareas; prioridades, fechas, categorías, marcar como completada	Alta
Gamificación	XP por acciones, niveles, rachas diarias, desbloqueo de funciones por nivel	Alta
Notificaciones	Alertas en tiempo real (WebSocket), recordatorios por email, avisos internos	Media
Chat en tiempo real	Mensajería entre amigos, grupos de estudio, colaboración de tareas	Media

Premium opcional	Desbloqueo inmediato de funciones, personalización adicional	Baja
------------------	--	------

4.2 Fuera del alcance

- Aplicación móvil nativa (iOS / Android).
- Integración con calendarios externos (Google Calendar, Outlook).
- Sistema de pagos en línea para el tier premium.
- Panel de administración avanzado.

4.3 Usuarios objetivo

- Estudiantes universitarios que requieren organizar actividades académicas.
- Personas en general que buscan mejorar sus hábitos de productividad personal.

5. Stack Tecnológico

Capa	Tecnología	Rol / Justificación
Backend	Java 17 + Spring Boot 3	Framework principal. REST API, seguridad, WebSocket.
	Spring Security + JWT	Autenticación y autorización stateless.
	Spring WebSocket / STOMP	Comunicación en tiempo real: notificaciones y chat.
	Spring Data JPA + Hibernate	ORM para acceso a base de datos.
	JavaMailSender	Envío de correos electrónicos de notificación.
Base de datos	MySQL 8	Persistencia relacional principal del sistema.
Frontend	HTML5 + CSS3 + JavaScript Vanilla	Interfaz de usuario sin frameworks pesados.
	jQuery + AJAX	Consumo de la API REST del backend de forma asíncrona.
	SockJS + STOMP.js	Cliente WebSocket para recibir notificaciones y chat.
Arquitectura	Hexagonal Architecture	Separación de dominio, puertos y adaptadores.
	Principios SOLID	Código mantenible, desacoplado y extensible.

Control de versiones	Git + GitHub	Gestión de código fuente y colaboración entre integrantes.
----------------------	--------------	--

5.1 Arquitectura del Sistema

El sistema seguirá una Arquitectura Hexagonal (Ports & Adapters), organizada en las siguientes capas:

- Domain: Entidades y reglas de negocio puras (Usuario, Tarea, Gamificación).
- Application: Casos de uso / servicios de aplicación que orquestan la lógica.
- Infrastructure: Adaptadores de entrada (REST Controllers, WebSocket Handlers) y salida (Repositorios JPA, Email, etc.).

El frontend se comunica exclusivamente con la capa de Infrastructure a través de la API REST (HTTP/JSON) y WebSocket (STOMP sobre SockJS).

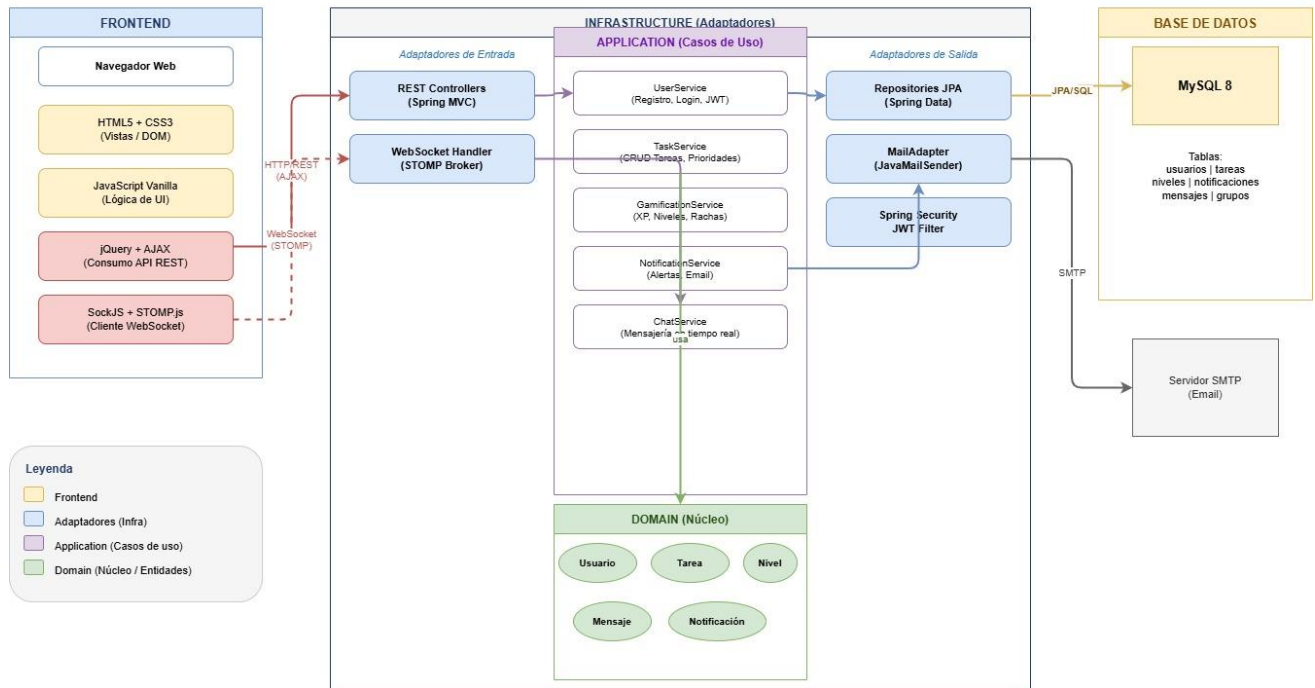
6. Diagramas del Sistema

6.1 Diagrama de Arquitectura General

A continuación se presenta la descripción textual de la arquitectura

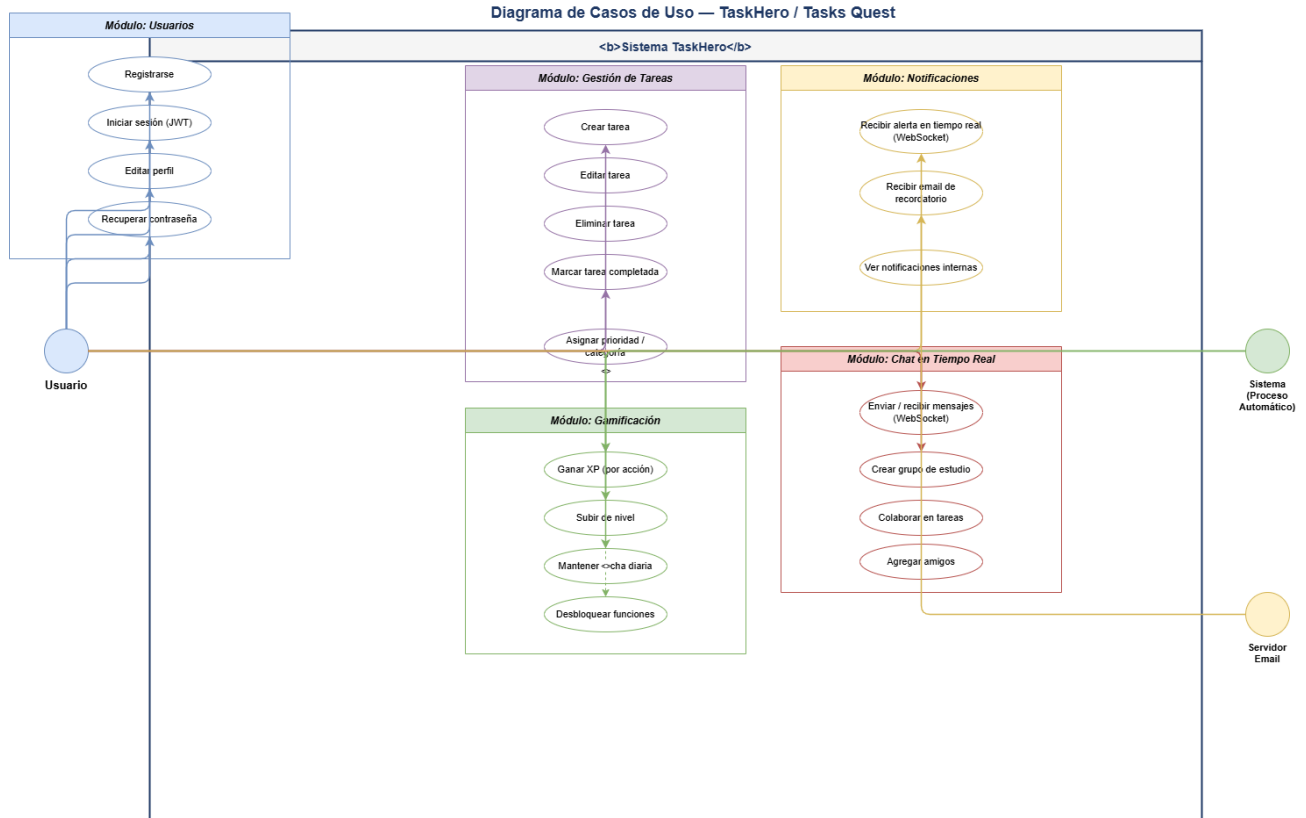
Componente Frontend	Componente Backend
HTML/CSS/JS Vanilla	Spring Boot Application
jQuery + AJAX → HTTP/REST	REST Controllers (Adaptadores de entrada)
SockJS + STOMP.js → WebSocket	WebSocket Handler (STOMP Broker)
Vistas (DOM dinámico)	Application Services (Casos de uso)
	Domain (Entidades + Reglas)
	Repositories JPA → MySQL 8

Arquitectura Hexagonal — TaskHero / Tasks Quest



6.2 Casos de Uso Principales

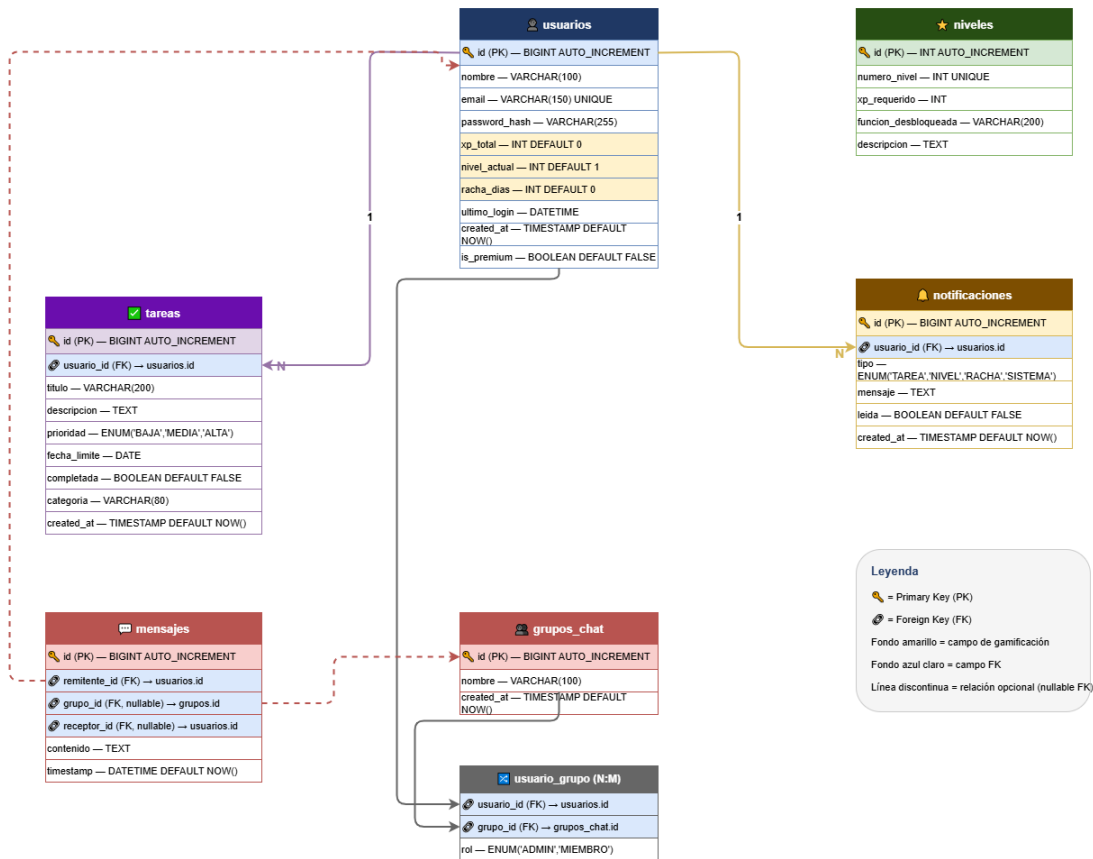
Actor	Caso de Uso	Descripción
Usuario	Registrarse / Iniciar sesión	Autenticación con JWT. Recuperación por email.
Usuario	Gestionar tareas	CRUD de tareas con prioridad, fecha y categoría.
Usuario	Completar tarea	Marca tarea como completada y gana XP.
Usuario	Subir de nivel	Acumula XP suficiente y desbloquea funciones.
Usuario	Recibir notificación	Alerta en tiempo real o por correo de eventos.
Usuario	Chatear en tiempo real	Envía/recibe mensajes con amigos o grupos.
Sistema	Calcular XP y rachas	Proceso automático al registrar actividad.



6.3 Modelo de Datos Principal (Entidades MySQL)

Entidad	Atributos clave	Relaciones
Usuario	id, nombre, email, password, xp, nivel, racha	1:N con Tarea, N:M con Amigo, 1:N con Notificación
Tarea	id, titulo, descripcion, prioridad, fechaLimite, completada, categoría	N:1 con Usuario
Nivel	id, numero, xpRequerido, funcionDesbloqueada	1:N con Usuario
Notificación	id, tipo, mensaje, leida, fechaCreacion	N:1 con Usuario
Mensaje (Chat)	id, contenido, timestamp, remitenteId, receptorId/grupoid	N:1 con Usuario
GrupoChat	id, nombre, miembros	N:M con Usuario

Diagrama Entidad-Relación — TaskHero / Tasks Quest (MySQL 8)



7. Sistema de Gamificación

7.1 Tabla de XP por Acción

Acción del Usuario	XP Ganado
Completar una tarea	10 XP
Mantener racha diaria	20 XP
Completar todas las tareas del día	50 XP
Iniciar sesión diariamente	5 XP
Cumplir una meta semanal	100 XP

7.2 Funciones Desbloqueables por Nivel

Nivel	XP Requerido	Función Desbloqueada
1	0 XP	Funciones básicas: crear tareas, notificaciones simples.
3	200 XP	Categorías personalizadas y prioridades avanzadas.
5	500 XP	Chat entre amigos habilitado.
8	1000 XP	Grupos de estudio y colaboración de tareas.
10	2000 XP	Estadísticas avanzadas y exportación de tareas.
Premium	Pago opcional	Acceso inmediato a todas las funciones.

8. Cronograma de Desarrollo (1 mes)

Actividad	Responsable	Semana	Estado
Análisis de requisitos y diseño de BD	Ambos	Sem. 1	Pendiente
Diseño de arquitectura hexagonal y entidades JPA	Stin Sánchez	Sem. 1	Pendiente
Estructura base del frontend (HTML/CSS/JS)	Marvin Castro	Sem. 1	Pendiente
Módulo de usuarios (registro, login, JWT)	Stin Sánchez	Sem. 1-2	Pendiente
Integración AJAX login/registro en frontend	Marvin Castro	Sem. 2	Pendiente
Módulo de gestión de tareas (CRUD REST)	Stin Sánchez	Sem. 2	Pendiente
Vistas de tareas con jQuery + AJAX	Marvin Castro	Sem. 2	Pendiente
Motor de gamificación (XP, niveles, rachas)	Stin Sánchez	Sem. 3	Pendiente
UI de gamificación (barra XP, niveles, logros)	Marvin Castro	Sem. 3	Pendiente
WebSocket: notificaciones en tiempo real	Stin Sánchez	Sem. 3	Pendiente
Chat en tiempo real (SockJS + STOMP.js)	Ambos	Sem. 3-4	Pendiente

Notificaciones por email (JavaMailSender)	Stin Sánchez	Sem. 4	Pendiente
Pruebas integrales y corrección de errores	Ambos	Sem. 4	Pendiente
Documentación técnica y presentación	Ambos	Sem. 4	Pendiente

9. Roles y Responsabilidades

Integrante	Rol	Responsabilidades principales
Stin Sánchez	Backend Developer	API REST con Spring Boot, arquitectura hexagonal, base de datos MySQL, seguridad JWT, WebSocket server, notificaciones por email, motor de gamificación.
Castro Rodríguez Marvin	Frontend Developer	Interfaces HTML/CSS/JS, consumo de API con jQuery + AJAX, cliente WebSocket con SockJS + STOMP.js, diseño UX/UI del sistema.

10. Conclusión

TaskHero / Tasks Quest representa una propuesta sólida y técnicamente justificada para el desarrollo de un minisistema web en el contexto de la materia Construcción de Software. La combinación de gestión de tareas con gamificación responde a una problemática real de accesibilidad y motivación en las herramientas de productividad actuales.

El stack tecnológico seleccionado (Java + Spring Boot, MySQL, JS Vanilla + jQuery + AJAX) permite aplicar de manera práctica los principios de la Arquitectura Hexagonal y los principios SOLID, garantizando un sistema escalable y mantenible. La distribución de roles entre backend y frontend facilita el trabajo paralelo y el cumplimiento de objetivos dentro del plazo de un mes establecido.

El presente anteproyecto establece una base clara y documentada para el inicio del desarrollo, con objetivos medibles, alcance definido, tecnologías justificadas, diagramas de referencia y un cronograma realista ajustado al tiempo disponible.